

Lab 4 - EECS 211

Winter 2026

1 Overview

In this assignment you will explore **stack buffer overflow vulnerabilities** on a bare-metal RISC-V system running in QEMU. You will:

1. Study the RISC-V calling convention and stack frame layout.
2. Trigger a buffer overflow and observe its effects using GDB.
3. Craft an exploit payload that hijacks control flow (ret2win).
4. Enable and evaluate a compiler-based defense (GCC stack protector).

This assignment uses the same bare-metal RISC-V + QEMU environment as HW1–HW3.

Lab environment: All tasks in this lab are performed on EECS-provided Linux servers.

Readings and references: Students are encouraged to consult the following resources for additional background:

- RISC-V ABI Specification — Official RISC-V calling convention and ELF psABI documentation.
- GCC Instrumentation Options — Documentation for `-fstack-protector-strong` and related options.

Rubrics (100 points): The code template is given on Canvas. You will upload one zip file that contains:

- **PDF report:** This includes screenshots and explanations for each task as described below.
- **exploit.py:** Your exploit script from Task 2.

Point breakdown:

- Task 1: 25 points
- Task 2: 35 points
- Task 3: 45 points (Part A: 10, Part B: 15, Part C: 15)

Points will be deducted if no screenshots or sufficient explanations are given. The students should also **fill the honesty statement and upload it to Canvas**, otherwise 30 points will be deducted.

Due date: 03/04/2026 23:59:59 PST

2 Environment Setup

Use the EECS server with the RISC-V toolchain. Set your PATH (add to your shell config if not already done in HW1–HW3):

```
# tcsh/csh:
setenv PATH /ecelib/eceware/riscv.old/bin/:$PATH

# bash/zsh:
export PATH=/ecelib/eceware/riscv.old/bin/:$PATH
```

2.1 Build and Run

```
make clean
make          # builds program.elf and program.img
make run      # runs in QEMU (blocks waiting for payload)
make run < payload.bin # runs with binary payload from stdin
```

2.2 Disassembly

These commands are useful for understanding the stack layout:

```
make dump      # shows objdump -d output (disassembly)
make nm        # shows symbol table sorted by address
```

3 GDB Debugging

GDB is essential for Tasks 1 and 2. Use a **two-terminal workflow**:

Terminal 1 — Start QEMU paused, waiting for GDB:

```
make qemu-gdb GDB_PORT=XXXX
```

Terminal 2 — Connect GDB:

```
make gdb GDB_PORT=XXXX
```

Replace XXXX with a unique port number (e.g., your student ID's last 4 digits) to avoid conflicts on the shared server. Both terminals must use the **same port number**.

To debug with a payload already piped in:

```
# Terminal 1:
make qemu-gdb-input GDB_PORT=XXXX # reads payload.bin via stdin
# Terminal 2:
make gdb GDB_PORT=XXXX
```

When GDB connects, the program is paused at the very first instruction. You should **set a breakpoint** at the function you want to inspect before typing **continue**. For example:

```
(gdb) break vuln_copy
(gdb) continue
```

3.1 Useful GDB Commands

See Table 1.

Command	Description
continue	Resume execution
stepi / nexti	Step one instruction
info registers	Show all register values
info frame	Show current stack frame details
print &buf	Print the address of buf
print \$sp	Print the stack pointer
print \$ra	Print the return address register
print \$s0	Print the frame pointer (s0/fp)
x/Ngx ADDR	Dump N 8-byte values at ADDR (e.g., x/14gx \$sp)
disas vuln_copy	Disassemble the vuln_copy function
break FUNCTION	Set a breakpoint

Table 1: Useful GDB commands for this assignment

4 Starter Code

The provided bare-metal program does the following:

1. Boots on QEMU “virt” machine, sets up stack, jumps to `main()`.
2. Prints the address of `win()` (the target function for your exploit).
3. Reads a **binary payload** from stdin via UART:
 - First 4 bytes: little-endian length `L`
 - Next `L` bytes: payload data
4. Calls `vuln_copy(payload, L)`, which copies `L` bytes into a **64-byte local buffer** on the stack — with **no bounds check**.
5. If the function returns normally, prints a “returned normally” message.

4.1 Key Files

File	Purpose
<code>src/vuln.c</code>	Vulnerable function + <code>win()</code> target + <code>main()</code>
<code>exploit.py</code>	Exploit template — fill in the blanks (Task 2)
<code>src/start.S</code>	Boot entry point — sets stack pointer, calls <code>main</code>
<code>src/uart.c</code>	Minimal UART driver for serial I/O
<code>src/util.c</code>	Hex printing, byte reading, <code>memcpy_unsafe</code>
<code>src/stack_protector.c</code>	GCC stack protector stubs (used in Task 3)
<code>linker.ld</code>	Linker script — places code at <code>0x80000000</code> , reserves stack
<code>Makefile</code>	Build system

Table 2: Starter code files

4.2 Payload Binary Format

Your exploit script must output a binary blob in this format:

[4 bytes: little-endian length `L`] [L bytes: payload data]

Example in Python:

```
import struct, sys
payload = b"AAAA" # your payload bytes
blob = struct.pack("<I", len(payload)) + payload
sys.stdout.buffer.write(blob)
```

5 Task 1 — Trigger and Characterize the Overflow

5.1 Step 1: Normal execution

Write a small Python script that generates a payload with a short length (e.g., 16 bytes of `'A'`). Run it:

```
python3 my_script.py > payload.bin
make run < payload.bin
```

Observe the normal output. The program should print “returned normally.”

5.2 Step 2: Trigger the overflow

Increase the payload length beyond the 64-byte buffer. Try different sizes (e.g., 72, 80, 88, 96 bytes). Observe what happens — you may see the program hang, crash, or jump to unexpected addresses.

5.3 Step 3: Use GDB to analyze

Launch the GDB debugging session (see Section 3). Set a breakpoint at the vulnerable function and continue execution.

1. **Before the copy:** Examine the stack frame. Note:

- The address of `buf` (`print &buf`)
- The stack pointer (`print $sp`)
- The frame pointer (`print $s0`)
- The saved return address on the stack (use `info frame` to find where `ra` is saved, then `x/gx ADDR` to read it)
- Dump the entire stack frame: `x/Ngx $sp` (choose `N` large enough to see all saved values)

2. **After the copy:** Step past the `memcpy_unsafe` call (or set another breakpoint after it). Examine the same memory locations. Identify which values were overwritten by your overflow payload.

5.4 Deliverable

In your report, include:

- Screenshots of GDB output showing the stack **before** and **after** the overflow.
- An explanation of the stack layout: where `buf`, saved `s0`, and saved `ra` are located relative to each other.
- What memory is corrupted when the payload exceeds 64 bytes.

6 Task 2 — Exploit: ret2win

Goal: Craft a payload that overwrites the saved return address so that when `vuln_copy` returns, execution jumps to `win()` instead of back to `main()`.

6.1 Step 1: Find the offset

Using GDB, determine the exact number of bytes from the start of `buf` to the saved return address (`ra`) on the stack. You already gathered the relevant addresses in Task 1 — now use them to compute the offset.

Hint: You can also use GDB's `disas` command to study the function assembly if needed.

6.2 Step 2: Find the address of `win()`

The program prints the address of `win()` at startup. You can also look it up inside GDB:

```
(gdb) print win
```

6.3 Step 3: Write your exploit

A template `exploit.py` is provided in the starter code. Open it and fill in the two values you determined in Steps 1 and 2:

- `OFFSET_TO_RA` — the number of padding bytes from `buf` to the saved `ra`
- `win_addr` — the address of `win()`

The template already handles building the payload (padding + little-endian address) and writing it in the correct binary format.

Important: RISC-V addresses are 64-bit (8 bytes). The template uses `struct.pack("<Q", address)` for little-endian encoding.

6.4 Step 4: Test

```
python3 exploit.py > payload.bin
make run < payload.bin
```

Success condition: The program prints:

```
*** WIN(): control flow hijacked! ***
```

6.5 Deliverable

- exploit.py — your exploit script
- In your report: explain how you determined the offset and the target address. Include GDB or disassembly evidence.

7 Task 3 — Defense: GCC Stack Protector

7.1 Part A: Enable the defense

The starter code includes `src/stack_protector.c`, which provides bare-metal stubs for GCC's stack protector. Rebuild with the stack protector enabled:

```
make clean
make SP=1
```

This adds `-fstack-protector-strong` to the compiler flags. Run your Task 2 exploit again:

```
make run < payload.bin
```

Expected result: Instead of jumping to `win()`, the program should print:

```
*** STACK SMASHING DETECTED (__stack_chk_fail) ***
```

Include a screenshot in your report showing the detection.

7.2 Part B: Analyze the defense mechanism

Compare the disassembly of `vuln_copy` with and without the stack protector:

```
# Without stack protector:
make clean && make
make dump      # save/screenshot the vuln_copy disassembly

# With stack protector:
make clean && make SP=1
make dump      # save/screenshot the vuln_copy disassembly
```

In your report, answer:

1. What extra instructions does the compiler insert at the **beginning** (prologue) and **end** (epilogue) of `vuln_copy` when `-fstack-protector-strong` is enabled?
2. What is the **canary value** (`__stack_chk_guard`)? Where is it stored on the stack relative to `buf` and the saved `ra`?
3. How does the compiler **detect** that a buffer overflow occurred?

7.3 Part C: Discussion

In your report, discuss:

1. Can an attacker **bypass** the stack canary? How?
2. What are the **performance costs** of stack protector instrumentation?
3. Name **one other defense mechanism** against buffer overflows (e.g., ASLR, non-executable stack / W^X, shadow stack). Briefly explain how it works and whether it would address the weakness of stack canary?